

Towards Decentralised Dynamic Reconfiguration of Software Systems

Mina Yavari and Damian Arellanes

School of Computing and Communications, Lancaster University, United Kingdom

Abstract. As software systems become more complex, researchers are looking for novel approaches to manage changes at scale without human intervention. As a result, on the one hand, dynamically reconfigurable software systems are designed to automatically adjust to environmental perturbations to help systems stay reliable even in uncertain situations. On the other hand, decentralised software systems are promising because they make it easier to manage large complex systems and they fit well in emergent paradigms like the Internet of Things. Most existing reconfiguration approaches rely on a central control to perform runtime changes, leading to well-known issues in terms of robustness and performance bottlenecks. In this paper, we introduce the notion of a Software Architecture Generator which enables the dynamic selection of concrete software architectures at runtime. Adaptive selection is performed in a decentralised manner using a hierarchical control structure that relies on protocols for starting reconfiguration as well as updating and aggregating weights without any exogenous, intrusive, centralised entity (e.g., a feedback control loop). Instead, software architectures themselves are equipped with multiple control entities which collectively find the optimal software architecture configuration that best adapts to the current operation environment.

Keywords: Dynamic reconfiguration · decentralised software systems · Software Architectures.

1 Introduction

Modern software is becoming more complex and is used in constantly changing environments. This has led to research in autonomic, self-adaptive and self-organising software systems [1, 2], focusing on making software reconfigurable and improving its ability to adjust to changing conditions at runtime.

In general, there are two main classifications in software systems to dynamic reconfiguration [3]. In the first one, the system configuration is modified based on an external control. This is known as exogenous (centralised) reconfiguration. Exogenous reconfiguration relies on an external coordinator, which introduces potential reliability issues. If the coordinator fails, it can lead to system failures, delays or incorrect adaptations. These issues are addressed by endogenous (decentralised) reconfiguration in which system components adjust their own

configuration to meet changing requirements and collaborate to adapt the system. Over the years, decentralised software systems have become more and more popular. Their design helps large and complex systems to run when subsystems are distributed in different locations. They are also fundamental to modern technologies like the Internet of Things (IoT), Edge computing and Cyber-Physical systems [4]. In this paper, decentralised reconfiguration is about the absence of a single global controller responsible for orchestrating architectural reconfigurations. Instead, decision-making is collectively performed by a hierarchy of controllers that communicate directly (not necessarily in a distributed fashion).

This paper proposes a framework for dynamic selection through what we call a Software Architecture Generator (SAG). A SAG defines a formal structure that hierarchically organises alternative software architectures and their components. Based on monitored environmental data (such as latency and memory), optimal architectural instances are selected using a decentralised hierarchical process comprising three reconfiguration protocols for initiation, updating and aggregation. The contribution of this work is providing a formal and decentralised planning framework for architectural reconfiguration that avoids centralised decision-making by distributing architectural selection across cooperating controllers operating on local information. Separating reconfiguration into three protocols provides a structured basis for architectural decision-making. This positions the work as a formally grounded alternative to existing centralised or loosely decentralised reconfiguration approaches.

The remainder of the paper is structured as follows. Section 2 reviews related work. Section 3 introduces a running example. Section 4 details our decentralised reconfiguration approach. Section 5 describes a validation and Section 6 presents the concluding remarks and future work.

2 Related Work

A weighted decision graph to guide model transformations for service level adaptation in an online shopping system is introduced in [5]. Authors in [6] present the concept of emergent software, where architectures are assembled at runtime using online, unsupervised learning. Its demonstration framework [7] contrasts manual and automated reconfiguration, but still operates within a centralised runtime system. In comparison, our work focuses specifically on the planning phase of reconfiguration and introduces a new formal mechanism for selecting software architectures. We propose an approach in which decision-making is done across components, without relying on machine learning or a central coordinator.

Decentralised dynamic reconfiguration focuses on enabling software systems to adapt their structure without relying on a central controller. A decentralised mechanism for adapting service-based applications in the cloud using distributed coordination strategies is presented in [8]. Moreover, an agent-based approach where components collaborate to monitor the system, manage resources and apply reconfigurations is proposed in [9]. Authors in [10] introduce a set of design patterns for managing multiple interacting MAPE-K loops, supporting decen-

tralised control in large-scale systems. Finally, component-based architecture using aspect-oriented programming and custom languages to support modular and decentralised reconfiguration is introduced in [11]. Compared to these, our approach emphasises formal decentralised planning.

Formal approaches to dynamic reconfiguration have employed rewriting rules [12, 13], logic-based reasoning [14] and algebraic techniques to describe evolving architectures. While these approaches offer structural clarity and correctness guarantees, they often assume a global system view or a centralised controller. Our work maintains formal rigour but is designed for decentralised execution, particularly supporting more scalable and distributed deployment scenarios.

3 Running example

Large-scale component-based systems can be designed using microservices, where each microservice performs a specific task. A common architectural pattern for these systems is the Single-Head-Single-Tail (SHST) pattern [15]. To explain how DRSS works in a real SHST situation, we use an example inspired by Amazon’s order processing where services such as handling the request, checking inventory, processing payment, shipping, tracking delivery and sending notifications are involved. These services are represented as components V_1 (head) through V_{11} (tail) and their structure is simplified through a SAG that shows joint sets of alternative components. As shown in Figure 1(a), the flow of the order starts with V_1 receiving the request, then proceeds through inventory checks by one of V_2 , V_3 or V_4 , payment processing at V_5 , shipping via V_6 or V_7 , delivery tracking with V_8 , V_9 or V_{10} and finally order confirmation through the notification service V_{11} . Some stages have multiple alternatives and DRSS dynamically selects among them to optimise system performance.

In Figure 1(a), each dotted box, called joint set, offers multiple implementations of the same functionality but with different non-functional requirements (NFRs). Inside each blue box, there is a vector showing latency and memory usage values. These values, expressed as vector components, are weights assigned to each NFR. Although the number of NFRs can be any number greater than zero, we consider only two here. Note that some components (V_1 , V_5 , V_{11}) are fixed with a vector of $(1, 1)$ since there are no alternatives to compare with and choose from.

The key idea is that the system’s architecture changes dynamically in response to environmental conditions like memory, availability or traffic load. For example, during peak traffic, the system might select lighter components such as V_3 , V_7 and V_{10} to conserve memory and handle many requests simultaneously.

4 Decentralised Reconfiguration

In this section, we present key concepts that are essential for the rest of the paper. For the sake of it, we treat software components as vectors which are organised in the form of joint sets within a SAG via the function μ (see Definitions 1

and 2). In Definition 2, there is only one component in V that has no joint set assigned to it (called bottom) and a single joint set in $\mathcal{J}$ that does not have any corresponding component (called head). The bottom represents the starting point of reconfiguration. Figure 1(d) illustrates the process of defining a SAG in three steps. First, components are defined; second, a set of joint sets is defined to group alternative components and, third, a mapping function μ assigns each component in V to the corresponding joint set, forming the structure of a SAG for dynamic architectural control. An id function Ω is defined for joint sets.

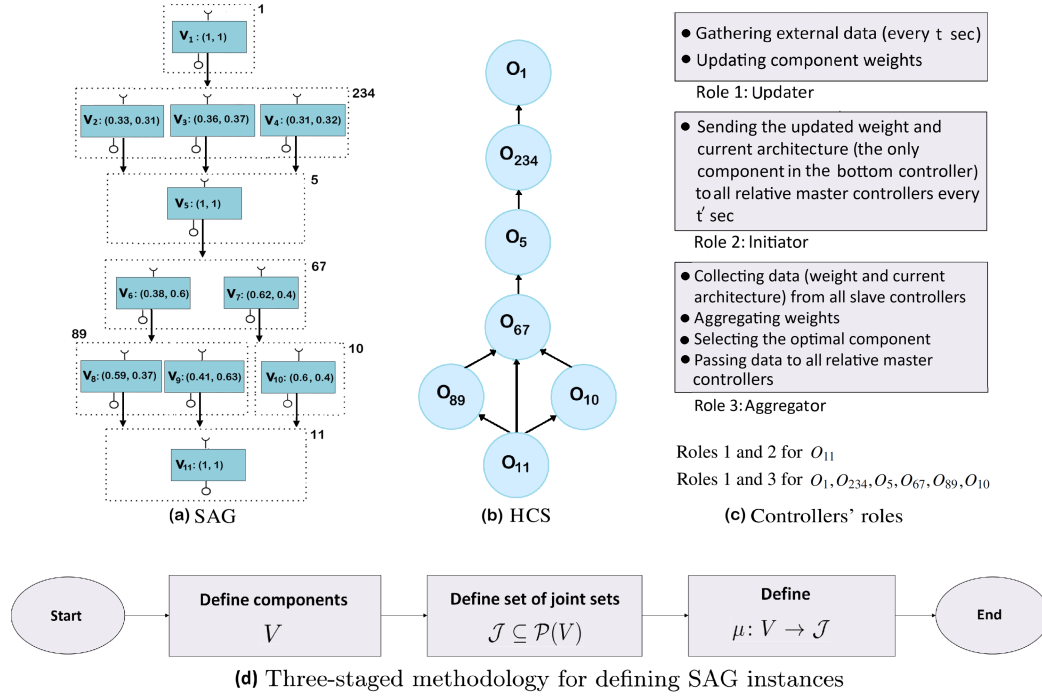


Fig. 1. SAG vs HCS and controllers' roles

Definition 1 (Component). A component is a vector of real numbers, where each element denotes the preference of some non-functional requirement.

Definition 2 (Software Architecture Generator (SAG)). A SAG is a quintuple $(V, \mathcal{J}, \mu, \Omega, n)$ where:

- V is a non-empty finite set $\{V_i\}_{i \in \mathbb{Z}^+}$ of n -dimensional components, equipped with the order relation $<$ given by $v_i < v_j \iff i \leq j$ for all $v_i, v_j \in V$,
- $\mathcal{J} \subseteq \mathcal{P}(V)$ is a set where $\mathcal{P}$ denotes power set and for each $\gamma \in \mathcal{J}$, called a joint set, we have $\sum_{k=\min I}^{\max I} k = \frac{(\max I + \min I)(\max I - \min I + 1)}{2} = \sum_{k \in I} k$ where $I = \{i \in \mathbb{Z}^+ | v_i \in \gamma\}$,

- $\mu: V \rightarrow \mathcal{J}$ is a non-surjective total function where $\exists! v \in V$ with $\mu(v) = \emptyset$, $\exists! \gamma \in \mathcal{J}$ with $\gamma \notin \mu(V)$ and, if R is the relation given by μ , $\forall (v, \gamma) \in R$, $\nexists (v', \gamma') \in R, v \in \gamma'$ and
- $\Omega: \mathcal{J} \rightarrow \mathbb{Z}^+$ is an id function for joint sets given by $\Omega(\gamma) = i \times 10^{|\gamma|-1} + (i+1) \times 10^{|\gamma|-2} + \dots + (i+|\gamma|-1) \times 10^0$ where $\gamma \in \mathcal{J}$ with $i = \min\{i \in \mathbb{Z}^+ | v_i \in \gamma\}$.

After constructing a SAG, a Hierarchical Control Structure (HCS) is formed (e.g., Figure 1(b)). A HCS is a master-slave relationship that identifies how joint set ids are connected to each other via a master function m and a slave function s . The main purpose of a HCS is to deal with joint set ids since, programmatically, it is more efficient to compute identifiers than joint sets themselves. Nevertheless, ω maps from identifiers to joint sets. After creating the master-slave relationship, one component from each joint set is selected to use as the initial state. A state function assigns a selected component to each joint set identifier.

Definition 3 (Hierarchical Control Structure (HCS)). A HCS is an 8-tuple $(V, \mathcal{J}, \mu, G, m, s, \delta, \omega)$ where:

- V is a set of components,
- $\mathcal{J} \subseteq \mathcal{P}(V)$ is a set where $\mathcal{P}$ denotes power set and for each $\gamma \in \mathcal{J}$, called a joint set, we have $\sum_{k=\min I}^{\max I} k = \frac{(\max I + \min I)(\max I - \min I + 1)}{2} = \sum_{k \in I} k$ where $I = \{i \in \mathbb{Z}^+ | v_i \in \gamma\}$,
- $\mu: V \rightarrow \mathcal{J}$ is a non-surjective total function where $\exists! v \in V$ with $\mu(v) = \emptyset$ and $\exists! \gamma \in \mathcal{J}$ with $\gamma \notin \mu(V)$,
- $G \subset \mathbb{Z}^+$ is a set of joint set identifiers,
- $m: G \rightarrow \mathcal{P}(G)$ is a master function that assigns a set of master ids to each master id, such that there exists exactly one $i \in G$ satisfying $m(i) = \emptyset$,
- $s: G \rightarrow \mathcal{P}(G)$ is a slave function that assigns a set of slave ids to each id, such that there exists exactly one $i \in G$ satisfying $s(i) = \emptyset$,
- $\delta: G \rightarrow V$ is a state function and
- $\omega: G \rightarrow \mathcal{J}$ maps each joint set id to its corresponding joint set.

For each joint set id, a controller is defined, which is a construct that contains data about a specific joint set (see Definition 4). Controllers in the HCS contain information about the list of components in its corresponding joint set, which controllers act as its masters, which as its slaves and a mapping between its components and their related joint sets. It also stores a weight for each component. Controllers in a HCS can be deployed on a single or multiple (distributed) machines. HCS partitioning/deployment is out of the scope of this paper.

Definition 4. A controller o is a 8-tuple $(G, M, S, V, \mu, n, i, \sigma)$ where:

- $G \subset \mathbb{Z}^+$ is a set of joint set identifiers,
- $M \subset G$ is a finite set of joint set ids that identify master ids,
- $S \subset G$ is a finite set of joint set ids that identify slave ids,
- V is a finite set of components,
- $\mu: V \rightarrow G$ is a non-surjective total function where $\exists! v \in V$ with $\mu(v) = \emptyset$ and $\exists! \gamma \in \mathcal{J}$ with $\gamma \notin \mu(V)$,

- $n \in \mathbb{N}$ is the fixed dimension of all the components in V ,
- $i \in \mathbb{Z}^+$ is the id of the controller o and
- $\sigma: V \rightarrow \mathbb{R}$ is a state function that assigns a real-valued number to each component.

Algorithm 1 handles deployment requests in a host machine by receiving controller instances, creating shared memory (for each controller) and starting three protocols: (i) initiating reconfiguration (i.e., Algorithm 5), (ii) "start-update-thread" (i.e., Algorithm 2) to gather external data and update component weights and (iii) "startAggregationThread" (i.e., Algorithm 3) to collect data from slave controllers, select the optimal component and pass data to the relatives masters. Algorithm 4 defines the behaviour for choosing an optimal component. Algorithm 2 is designed to update the component's state (weight) by using environmental data every t seconds. After collecting the new states, it updates the shared memory for the threads of a controller. There is no shared memory to exchange data among controllers.

Algorithm 1: Manager (Input: Controller: $O = (G, M, S, V_o, \mu_o, n, i_o, \sigma)$)

```

while 1
  O := receiveController()
  sharedMem := new-shared-memory( $G, M, S, V_o, \mu_o, n, i_o, \sigma$ )
  startUpdateThread(sharedMem, 2)
  startAggregationThread(sharedMem)
  if sharedMem( $S$ ) =  $\emptyset$  then startInitiatorThread(sharedMem, 10)
```

For a controller with no slaves, the manager assigns the protocol "startInitiatorThread" (i.e., Algorithm 5) which finds the optimal component at the bottom of the HCS and sends this updated information to all master controllers. To make the protocols easier to understand, we assign three distinct roles to each controller, as illustrated in Figure 1(c): Role 1 corresponds to the start-update-thread protocol, Role 2 to the startInitiatorThread protocol and Role 3 to the startAggregationThread protocol. Role 1 gathers external environmental data (every t seconds) and updates the weights of its associated components. Role 2 operates only on the bottom controller and is responsible for sending the updated weights and current architecture (i.e., the single component of the bottom controller) to all relevant master controllers at fixed intervals (every t' seconds). Role 3 collects weights and current optimal architectures from all relative slave controllers, aggregates these weights, selects the optimal component and passes the result to the relevant masters.

Algorithm 3 allows a master controller to collect data from its slaves. When it receives data from all them, it computes compound weights for each component and then chooses the one with the minimum value by Algorithm 4. The chosen component is then appended to the previously selected components to update the optimal architecture. After that, the master controller sends the new optimal architecture and the new compound weight to its relative masters.

Algorithm 2: startUpdateThread (Input: sharedMem, t)

```

while 1
  sleep( $t$ );  $e := \text{getVectorFromExtEnv}()$ 
  lock sharedMem
     $state' := \emptyset$ 
    for  $v \in \text{sharedMem}(V_o)$ 
       $state' := state' \cup \{(v, v.e)\}$ 
    update_hared_memory(sharedMem,  $\sigma$ ,  $state'$ )
  release sharedMem

```

Algorithm 3: startAggregationThread (Input: sharedMem)

```

count_slaves := 0; slaveArchWeight :=  $\emptyset$ 
while 1
  (slaveId, slaveArch, slaveWeight) := receive_from_slave()
  count_slaves := count_slaves + 1
  slaveArchWeight := slaveArchWeight  $\cup \{(slaveId, slaveArch, slaveWeight)\}$ 
  lock sharedMem
  if count_slaves = sharedMem( $S$ ).size()
    (arch, weight) := chooseOptimalComponent(sharedMem, slaveArchWeight)
    for master in sharedMem( $M$ )
      send(sharedMem( $i_o$ ), arch, weight)
    count_slaves := 0; slaveArchWeight :=  $\emptyset$ 
  release sharedMem

```

5 Validation

To make the whole process clear, let us consider the running example of Amazon's order processing. In Role 1, every t seconds, component weights are updated (e.g., with environmental weights (0.54, 0.67), V_{11} updates to $0.54 \times 1 + 0.67 \times 1 = 1.21$). In Role 2, the initiator sends the updated weights and architecture (V_{11}) to its masters (O_{89} and O_{10}). In Role 3, each master aggregates the received weights with its own component's weights to select the optimal one. For example, O_{89} compares V_8 ($0.54 \times 0.59 + 0.67 \times 0.37 \approx 0.57$) and V_9 ($0.54 \times 0.41 + 0.67 \times 0.63 \approx 0.64$) and selects V_8 , forming the partial architecture $V_{11} \rightarrow V_8$ with weight $1.21 + 0.57 = 1.78$. If the current environmental weights for latency and memory are (0.5, 0.61), the weights of V_{10} is $0.5 \times 0.6 + 0.61 \times 0.4 \approx 0.54$ and O_{10} chooses V_{10} with compound weight $1.21 + 0.54 = 1.75$ which is sent to O_{67} . The calculations of O_{89} and O_{10} are performed simultaneously, with no dependency between them. Note that each aggregator uses the most recent environmental weights available when the calculation is done. This process continues until the top-level controller O_1 receives the data and makes the final selection. For instance, assuming O_{67} and O_{234} respectively choose V_6 and V_3 , the optimal configuration determined by O_1 is: $V_1 \rightarrow V_3 \rightarrow V_5 \rightarrow V_6 \rightarrow V_8 \rightarrow V_{11}$. This final configuration represents the optimal configuration under current environmental conditions.

To validate the proposed approach, we implemented it and made its source code available at <https://github.com/mstrn0682394262/Dynamic-Reconfiguration>.

Algorithm 4: chooseOptimalComponent (Input: sharedMem, slaveArchWeight)

```

optimalWeight :=  $\infty$ ; optimalCompArch := []
lock sharedMem
  for  $v_j$  in sharedMem( $V_o$ )
    aggWeight :=  $\pi_1(\text{slaveArchWeight}(\text{sharedMem}(\mu_o)(v_j))) + \text{sharedMem}(\sigma)(v_j)$ 
    if aggWeight  $\leq$  optimalWeight
      optimalWeight := aggWeight
      optimalCompArch :=  $\pi_0(\text{slaveArchWeight}(\text{sharedMem}(\mu_o)(v_j))) ++ [j]$ 
  release sharedMem
return (optimalCompArch, optimalWeight)

```

Algorithm 5: startInitiatorThread (Input: sharedMem, t)

```

while 1
  sleep( $t$ )
  lock sharedMem
    optimalArch :=  $\emptyset$ , optimalWeight :=  $\infty$ 
    for  $v_j \in \text{sharedMem}(V_o)$ 
      if  $\text{sharedMem}(\sigma)(v_j) \leq \text{optimalWeight}$ 
        optimalWeight :=  $\text{sharedMem}(\sigma)(v_j)$ 
        optimalArch :=  $[j]$ 
    for master in  $\text{sharedMem}(M)$ 
      send( $\text{sharedMem}(i_o)$ , optimalArch, optimalWeight)
  release sharedMem

```

6 Conclusion

This paper introduced the notions of SAG and HCS as well as three main protocols to realise decentralised reconfiguration of software architectures. Our approach groups software components into joint sets and manages them using a hierarchical control structure, allowing a system to adjust itself at runtime without the need of a central controller. To support runtime adaptation, the system triggers reconfiguration every t seconds via a well-defined protocol. The other two protocols are used to update component weights using environmental data and combine information from corresponding slaves, respectively. Once information is received, the component with the minimum compound weight is chosen and then appended to the currently selected software architecture. The new architecture information is then sent to all the corresponding masters for further decision making. This process is repeated until reaching the top-level component. To validate that our proposed method enables the dynamic selection of software architectures at runtime, we implemented and published it. This method is especially helpful for changable environments like cloud computing and the IoT, where systems need to be flexible and work without constant human control. In the future, we plan to evaluate the efficiency of our approach by analysing its execution time empirically. We also plan to validate its applicability in a real-world scenario.

References

1. De Sanctis, M., Bucchiarone, A., Marconi, A.: Dynamic Adaptation of Service-Based Applications: A Design for Adaptation Approach. *Journal of Internet Services and Applications* 11(1) (2020). <https://doi.org/10.1186/s13174-020-00123-6>
2. Arellanes, D.: Self-Organizing Software Models for the Internet of Things: Complex Software Structures That Emerge Without a Central Controller. In: *IEEE Systems, Man, and Cybernetics Magazine*, vol. 7, no. 3, pp. 4–9. IEEE (2021). <https://doi.org/10.1109/MSMC.2021.3062822>
3. Weyns, D., Haesevoets, R., Van Eylen, B., Helleboogh, A., Holvoet, T., Joosen, W.: Endogenous versus Exogenous Self-Management. In: *Proceedings of the 2008 International Workshop on Software Engineering for Adaptive and Self-Managing Systems*, pp. 41–48. ACM, Leipzig, Germany (2008). <https://doi.org/10.1145/1370018.1370027>
4. Kratzke, N., Quint, P.-C.: Understanding Cloud-Native Applications after 10 Years of Cloud Computing – A Systematic Mapping Study. *Journal of Systems and Software* 126, 1–16 (2017). <https://doi.org/10.1016/j.jss.2017.01.001>
5. Chen, B., Peng, X., Yu, Y., Nuseibeh, B., Zhao, W.: Self-Adaptation through Incremental Generative Model Transformations at Runtime. In: *Proceedings of the 36th International Conference on Software Engineering*, pp. 676–687. ACM, Hyderabad, India (2014). <https://doi.org/10.1145/2568225.2568310>
6. Rodrigues Filho, R., Porter, B.: Defining Emergent Software Using Continuous Self-Assembly, Perception and Learning. *ACM Transactions on Autonomous and Adaptive Systems* 12(3), 1–25 (2017). <https://doi.org/10.1145/3092691>
7. Rodrigues Filho, R., Porter, B.: Demonstrating a Runtime Machine-Centric Emergent Software Architecture Framework. In: *Proceedings of the 2016 IEEE International Conference on Autonomic Computing (ICAC)*, pp. 239–240. IEEE, Würzburg, Germany (2016). <https://doi.org/10.1109/ICAC.2016.35>
8. Nallur, V., Bahsoon, R.: A Decentralised Self-Adaptation Mechanism for Service-Based Applications in the Cloud. In: *Proceedings of the IEEE* (2012)
9. Saadi, A., Oussalah, M., Henni, A., Bennouar, D.: Handling the Dynamic Reconfiguration of Software Architectures Using Intelligent Agents. In: *Proceedings* (2015)
10. Weyns, D., Schmerl, B., Grassi, V., Malek, S., Mirandola, R., Prehofer, C., Wuttke, J., andersson, J., Giese, H., Götsch, K.: On Patterns for Decentralised Control in Self-Adaptive Systems. In: *Proceedings* (2013)
11. Baresi, L., Guinea, S., Tamburrelli, G.: Towards Decentralized Self-Adaptive Component-Based Systems. In: *Proceedings of the 2008 International Workshop on Software Engineering for Adaptive and Self-Managing Systems*, pp. 57–64. ACM, Leipzig, Germany (2008) <https://doi.org/10.1145/1370018.1370029>
12. Arellanes, D.: Composition Machines: Programming Self-Organising Software Models for the Emergence of Sequential Program Spaces. In: Chin, W.-N., Xu, Z. (eds.) *TASE 2024, LNCS*, vol. 14777, pp. 19–37. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-64626-3_2
13. Fernández, M., Mackie, I.: Hierarchical Higher-Order Port Graphs: A Rewriting-Based Modeling Language. *ACM* (2024)
14. Bozga, M., Bueri, L., Iosif, R.: Decision Problems in a Logic for Reasoning About Reconfigurable Distributed Systems. In: *IJCAR 2022, LNCS*, vol. 13385, pp. 659–677. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-10769-6_40
15. Xue, Gang, Shuiguang Deng, Di Liu and Zeming Yan. 2021. ‘Reaching Consensus in Decentralized Coordination of Distributed Microservices’. *Computer Networks* 187 (March): 107786. <https://doi.org/10.1016/j.comnet.2020.107786>.